Experiment -1

Write a MATLAB program to generate two sinusoidal signals using the equation

$$y(t) = \sin(2\pi f t)$$

where the frequencies are $f_1 = 2$ Hz and $f_2 = 10$ Hz. Consider the time range from $t = 0$ to $t = 1$ seconds with a step size of 0.01. Perform the following operations and plot the results using the subplot command:

1. Plot both sinusoidal signals.
2. Addition of the two signals.
3. Subtraction of the two signals.
4. Multiplication of the two signals.

Label the axes as time and amplitude, and give suitable titles for each plot.

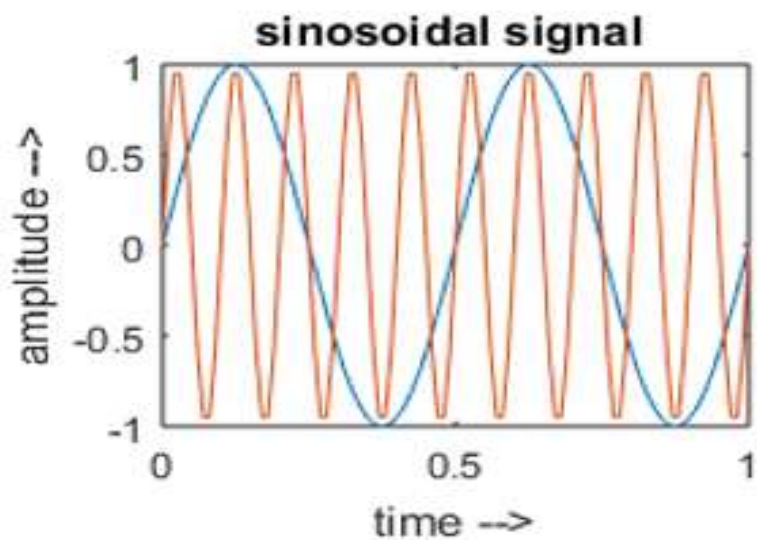
Aim: To generate two sinusoidal signals using MATLAB and perform addition, subtraction and multiplication operations on them and plot the resulting signals using subplots.

```
clc; clear all; close all;
t=0:0.01:1
y1=sin(2*pi*2*t)
y2=sin(2*pi*10*t)
a=y1+y2;
s=y1-y2;
m=y1.*y2;
subplot(2,2,1);
plot(t,y1,t,y2);
xlabel('time -->');
ylabel('amplitude -->');
title('sinosoidal signal');
```

$$y(t) = \sin(2\pi f t)$$

frequencies are $f_1 = 2$ Hz and $f_2 = 10$ Hz.

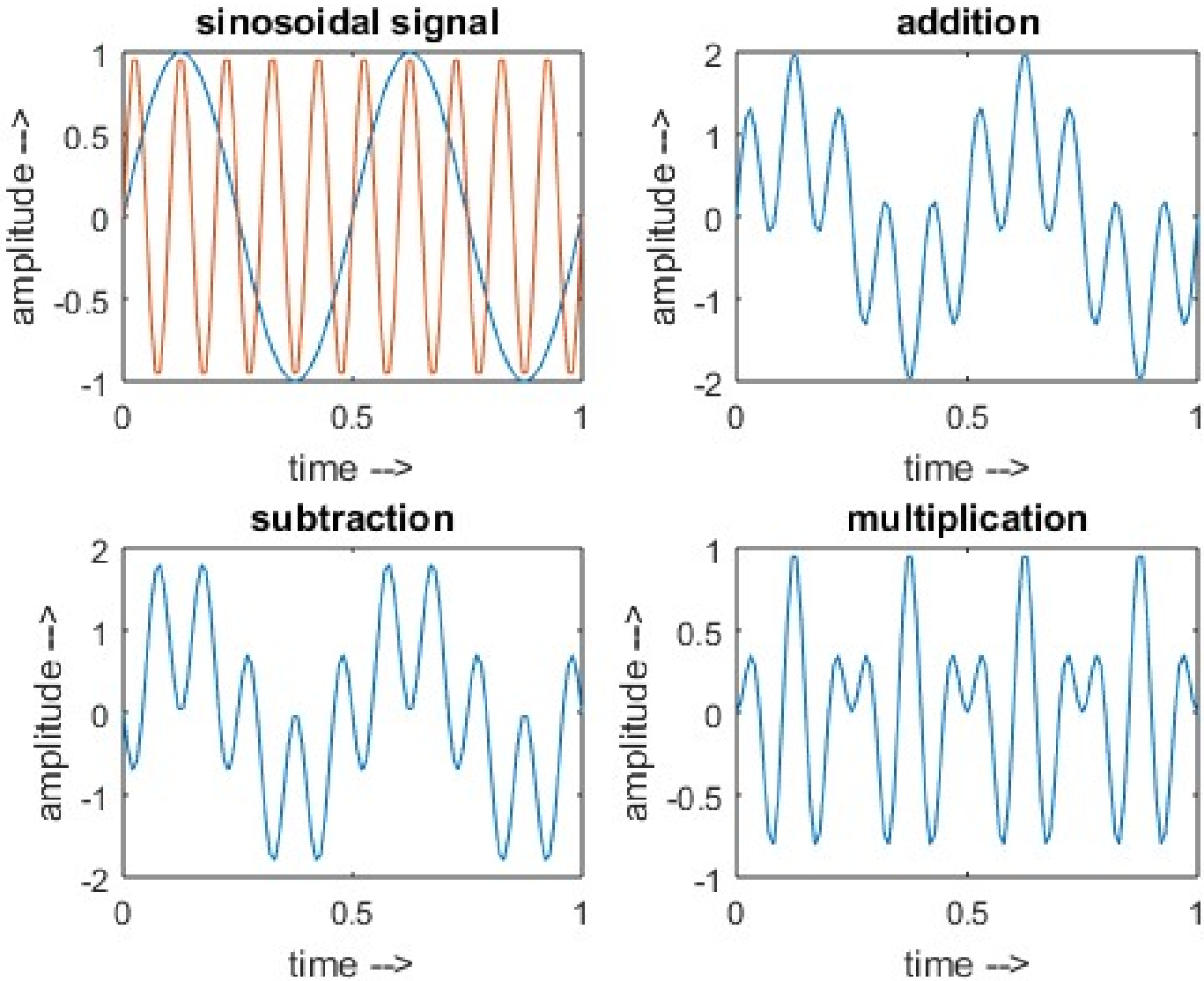
$t = 0$ to $t = 1$ seconds with a step size of 0.01.



```
subplot(2,2,2);
plot(t,a);
xlabel('time -->');
ylabel('amplitude -->');
title('addition');

subplot(2,2,3);
plot(t,s);
xlabel('time -->');
ylabel('amplitude -->');
title('subtraction');

subplot(2,2,4);
plot(t,m);
xlabel('time -->');
ylabel('amplitude -->');
title('multiplication');
```


sinosoidal signal
addition
subtraction
multiplication
amplitude -->
time -->
1
0.5
0
-0.5
-1
2
-2
0
0.5
1

Experiment -2

Write a MATLAB program to generate two sinusoidal signals using the equation

$$y(t) = \sin(2\pi f t)$$

where the frequencies are $f_1 = 2$ Hz and $f_2 = 10$ Hz. Consider the time range from $t = 0$ to $t = 1$ seconds with a step size of 0.01.

Perform the following operations using **separate user-defined function files:**

1. Addition of the two signals
2. Subtraction of the two signals
3. Multiplication of the two signals

Write a main MATLAB program to call these functions and plot the original signals and the resulting signals using the subplot command with proper axis labels and titles.

Main File:

```
clc; clear all; close all;
t=0:0.01:1;
y1=sin(2*pi*2*t);
y2=sin(2*pi*10*t);
[a,s,m]=process(y1,y2);
**Remaining lines are same as experiment-1
```

Sub file: (with file name **process.m)**

```
function [a,s,m] = process(y1,y2)
a=y1+y2;
s=y1-y2;
m=y1.*y2;
end
```

Experiment -3

Write a MATLAB program to generate a set of 50 data points using the linear model

$$y = -0.3x + 0.2r$$

where r is a normally distributed random number. Consider $x = 1, 2, 3, \ldots, 50$. Use MATLAB commands to determine the best linear fit using polynomial curve fitting.

```
clc;clear all;close all;
x=1:50;
y=-0.3*x+0.2*randn(1,50);
p=polyfit(x,y,1);
f=polyval(p,x)
plot(x,y,'o',x,f,'-')
legend('data','linear fit')
xlabel('X values --->');
ylabel('Y values --->');
title('Linear Curve Fitting');
```

randn(1,50) → Generates **50 random numbers with normal distribution**

<u>The **polyfit()** command is used to determine the **best fitting polynomial**.</u>

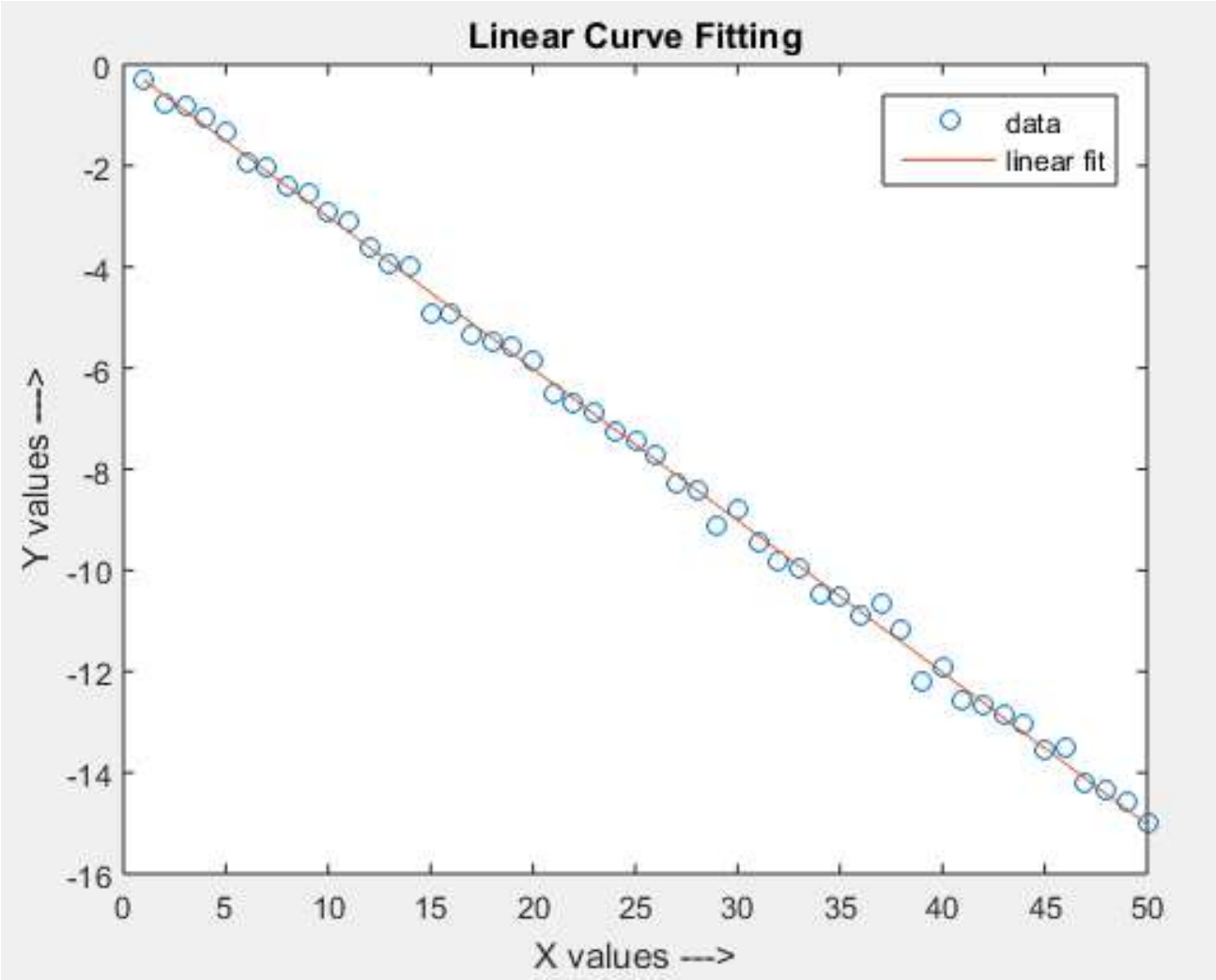
$$p = polyfit(x, y, n)$$

x → input data
y → output data
n → order of polynomial

<u>The **polyval()** command evaluates the polynomial.</u>

$$f = polyval(p, x)$$

This calculates the **predicted y values** using the fitted equation.
f → **represents the fitted line values**.


Linear Curve Fitting
data
linear fit
Y values --->
X values --->
0
-2
-4
-6
-8
-10
-12
-14
-16
0
5
10
15
20
25
30
35
40
45
50

Experiment -4

Write a MATLAB program to generate 10 equally spaced points in the interval 0 to 4π. Compute the sine values of these points and obtain a third-degree polynomial approximation using polynomial curve fitting. Evaluate the fitted polynomial over the interval 0 to 4π and plot the original data points and the approximated curve.

```
clc; clear all; close all;
x = linspace(0,4*pi,10);
y = sin(x);

p = polyfit(x,y,3);

x1 = linspace(0,4*pi);
f = polyval(p,x1);

plot(x,y,'o',x1,f,'-')
legend('data points','cubic fit')
xlabel('X values --->');
ylabel('Y values --->');
title('Polynomial Curve Fitting')
```

The **linspace()** function generates **equally spaced numbers** between two limits.

x=linspace(a,b,n) **a** → starting value **b** → ending value
n → number of points

<u>The **polyfit()** command is used to determine the **best fitting polynomial**.</u>

$$p = polyfit(x, y, n)$$

x → input data
y → output data
n → order of polynomial

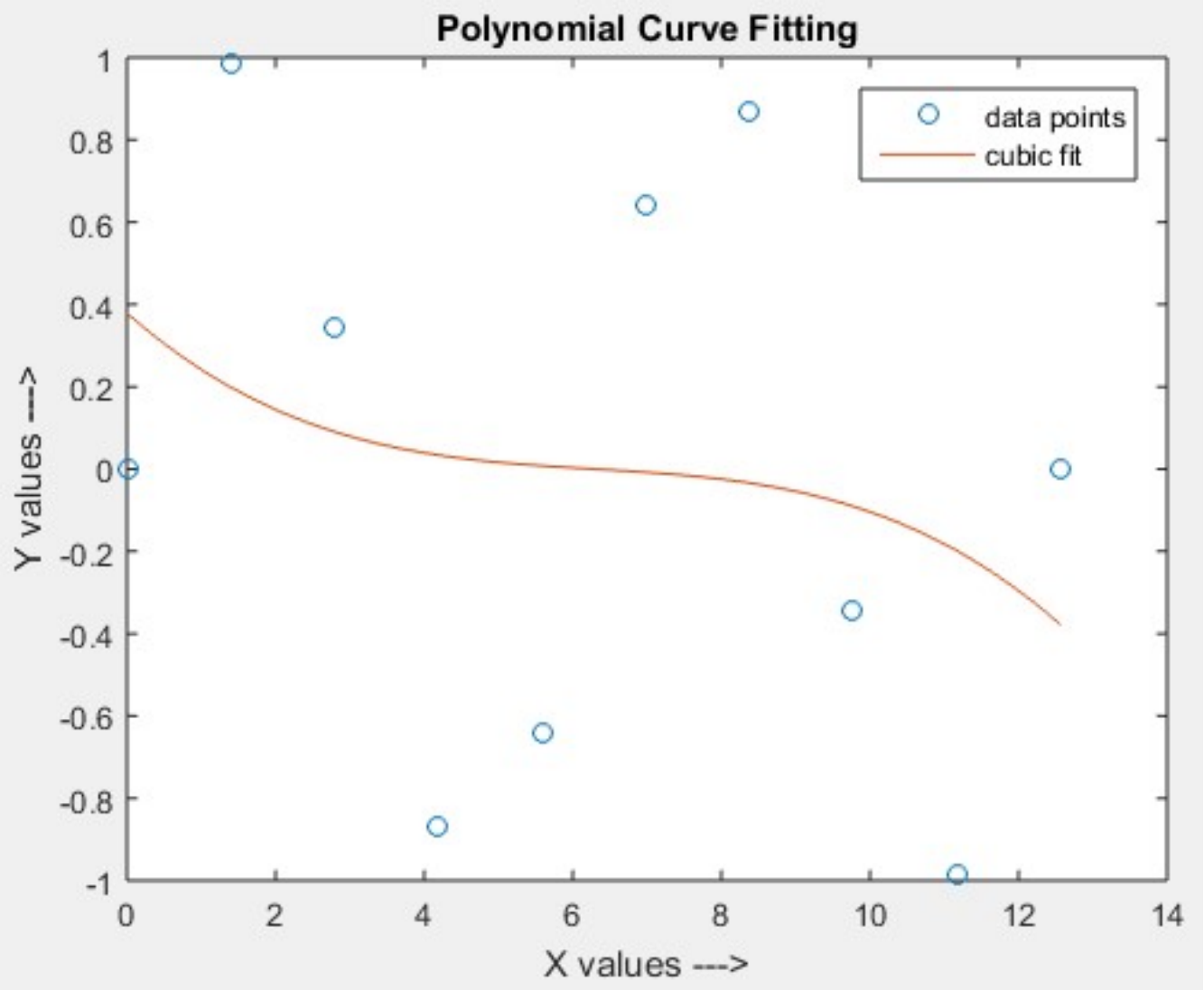
Cubic equation: Eg: $y = a_3x^3 + a_2x^2 + a_1x + a_0$

linspace() generates many points between **0 and 4π**.

<u>The **polyval()** command evaluates the polynomial.</u>

$$f = polyval(p, x)$$

This calculates the **predicted y values** using the fitted equation.
f → represents the fitted line values.

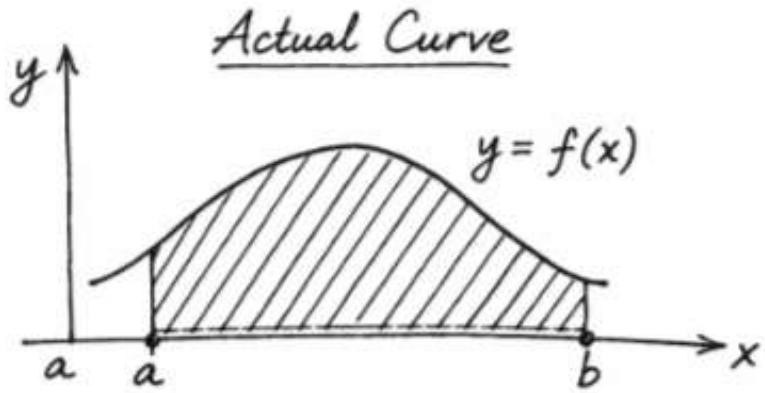

Polynomial Curve Fitting
data points
cubic fit
Y values --->
X values --->
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1
0
2
4
6
8
10
12
14

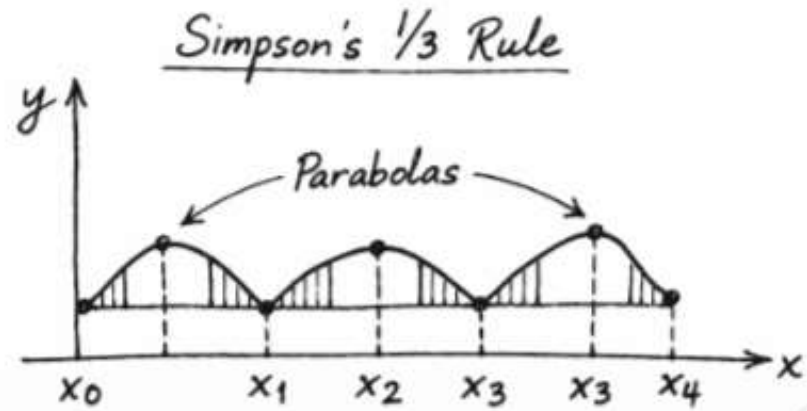
Experiment -5

Write a MATLAB program to evaluate the definite integral

$$I = \int_0^{\pi/2} x \sin(x)\, dx$$

using Simpson's $\frac{1}{3}$ rule with $n = 20$ subintervals. Display the computed value of the integral.


Actual Curve
y
y = f(x)
a
a
b
x


Simpson's 1/3 Rule
y
Parabolas
x
x0
x1
x2
x3
x3
x4

Simpson's $1/3$ Rule is a **numerical integration method** used to approximate the value of a definite integral when the exact integration is difficult. It approximates the curve between points using **parabolic arcs (quadratic polynomials)** instead of straight lines.

The interval $[a, b]$ is divided into **an even number of equal subintervals** n with step size

$$h = \frac{b-a}{n}$$

The formula used is

$$I = \frac{h}{3}\left[f(x_0) + f(x_n) + 4\sum f(x_{odd}) + 2\sum f(x_{even})\right]$$

```
clc;clear all;close all;
f=@(x)x*sin(x)
a=0;
b=pi/2;
n=20;
h=(b-a)/n;
s=f(a)+f(b);
for i=1:2:n-1
    s=s+4*f(a+i*h)
end
```

f=@(x)x*sin(x) defines the function using a **function handle**.

f(x)=xsin(x)

@(x) tells MATLAB that **f is a function of x**.

So when MATLAB executes **f(1)**, it evaluates, 1sin(1)

a and **b** represent the **limits of integration**.

Lower limit a=0

Upper limit b=π/2

$$I = \int_0^{\pi/2} x \sin(x)\, dx$$

n=20 specifies the number of **subintervals**.

Important rule for **Simpson's 1/3 rule**

h=(π/2−0)/20 calculates the **width of each subinterval**.

In **Simpson's rule**, the first and last terms are added separately s=f(a)+f(b)

Summation of odd terms: The loop calculates the **terms with coefficient 4**.

The index increases by **2**: I,e: 1,3,5,7,...,n−1

Each corresponding function value is multiplied by **4** according to Simpson's rule.

$$I = \frac{h}{3}\left[f(x_0) + f(x_n) + 4\sum f(x_{odd}) + 2\sum f(x_{even})\right]$$

```
for i=2:2:n-2
    s=s+2*f(a+i*h);
end
```

Summation of Eveen Terms: This loop calculates the **terms with coefficient 2**. The index increases by **2**:

2,4,6,8,...,n−2

Each function value is multiplied by **2**.

```
I=(h/3)*s;
display(I)
```

Experiment -6

Write a MATLAB program to numerically evaluate the definite integral

$$\int_0^5 (x^2 + 2x + 1)\, dx$$

using the **Trapezoidal Rule** with **10 equal subintervals**.

Mathematical Formula

$$\int_a^b f(x)\,dx \approx \frac{h}{2}\left[f(a) + f(b) + 2\sum_{i=1}^{n-1} f(x_i)\right]$$

Meaning of Terms

- a, b → Lower and upper limits
- n → Number of subintervals
- $h = \frac{b-a}{n}$ → Width of each interval
- $x_i = a + ih$ → Intermediate points
- $f(x)$ → Function to be integrated

```
clc;clear all;close all;
f = @(x) x^2 + 2*x + 1;
a = 0;
b = 5;
n = 10;

h = (b - a)/n;

sum = f(a) + f(b);

for i = 1:n-1
    x = a + i*h;
    sum = sum + 2*f(x);
end
```

$$\int_0^5 (x^2 + 2x + 1)\, dx$$

$$\int_a^b f(x)\, dx \approx \frac{h}{2}\left[f(a) + f(b) + 2\sum_{i=1}^{n-1} f(x_i)\right]$$

```
I = (h/2) * sum;

fprintf ('Integral using Trapezoidal Rule, I = %.6f\n', I);
```

`%.6f`

`%` → Indicates a **format specifier**

`.6` → Specifies **6 digits after the decimal point**

`f` → Stands for **floating-point number (decimal value)**

\n→ is the new line character.

Experiment -7

Write a MATLAB program to:

1. Find the **eigenvalues and eigenvectors** of a given matrix A
2. Find the **inverse of the matrix**
3. Verify that:

$$A \cdot A^{-1} = I$$

```
clc;
clear;
Close all;

A = input('Enter matrix A: ');

% Eigenvalues and Eigenvectors
[V, D] = eig(A);

disp('Eigenvalues (D):');
disp(D);

disp('Eigenvectors (V):');
disp(V);
```

```
% Inverse of matrix
A_inv = inv(A);

disp('Inverse of matrix A:');
disp(A_inv);

% Verification
I = A * A_inv;

disp('Verification (A * A_inv):');
disp(I);
```

Experiment -8

Solve the following system of linear algebraic equations using **Gauss Elimination Method** and obtain the solution vector using MATLAB:

$$6x + 3y + 2z = 6$$

$$6x + 4y + 3z = 0$$

$$20x + 15y + 12z = 0$$

$$6x + 3y + 2z = 6$$

$$6x + 4y + 3z = 0$$

$$20x + 15y + 12z = 0$$

Remove **x from 2nd & 3rd equations**

Then remove **y from 3rd equation**

Then solve from bottom

Write Augmented Matrix

$$\begin{bmatrix} 6 & 3 & 2 & | & 6 \\ 6 & 4 & 3 & | & 0 \\ 20 & 15 & 12 & | & 0 \end{bmatrix}$$

Pivot = 6 (first row) **Make first column below pivot ZERO**

$$R_2 = R_2 - R_1$$

$$[6\ 4\ 3 \mid 0] - [6\ 3\ 2 \mid 6]$$

$$= [0\ 1\ 1 \mid -6]$$

Make Row 3 first element zero

We need to remove **20**

$$R_3 = R_3 - \frac{20}{6} R_1$$

$$R_3 = [20\ 15\ 12 \mid 0] - \frac{20}{6}[6\ 3\ 2 \mid 6]$$

$$= [20\ 15\ 12 \mid 0] - [20\ 10\ 6.67 \mid 20]$$

$$= [0\ 5\ 5.33 \mid -20]$$

New Matrix

$$\begin{bmatrix} 6 & 3 & 2 & | & 6 \\ 0 & 1 & 1 & | & -6 \\ 0 & 5 & 5.33 & | & -20 \end{bmatrix}$$

Make second column below pivot ZERO

Pivot = 1 (Row 2)

Make Row 3 second element zero

$$R_3 = R_3 - 5R_2$$

$$[0\ 5\ 5.33 \mid -20] - 5[0\ 1\ 1 \mid -6]$$

$$= [0\ 5\ 5.33 \mid -20] - [0\ 5\ 5 \mid -30]$$

$$= [0\ 0\ 0.33 \mid 10]$$

Final Triangular Matrix

$$\begin{bmatrix} 6 & 3 & 2 & | & 6 \\ 0 & 1 & 1 & | & -6 \\ 0 & 0 & 0.33 & | & 10 \end{bmatrix}$$

Back Substitution

From last row:

$$0.33z = 10 \Rightarrow z = 30$$

Row 2:

$$y + z = -6$$

$$y + 30 = -6 \Rightarrow y = -36$$

Row 1:

$$6x + 3y + 2z = 6$$

$$6x + 3(-36) + 2(30) = 6$$

$$6x - 108 + 60 = 6$$

$$6x = 54 \Rightarrow x = 9$$

Final Answer

$$x = 9, \quad y = -36, \quad z = 30$$

Program Line	Explanation
clc; clear;	clc → clears command window clear → removes all previous variables
A = [6 3 2; 6 4 3; 20 15 12];	Coefficient matrix (3×3 system)
b = [6; 0; 0];	RHS column vector
Aug = [A b];	Augmented matrix → combines A and b
n = size(A,1);	size(A,1) → number of rows → number of equations

FORWARD ELIMINATION

Program Line	**Explanation**
for i = 1:n-1	i = pivot row Runs till n-1 (last row no need elimination)
for j = i+1:n	j = rows below pivot row
factor = Aug(j,i)/Aug(i,i);	Aug(j,i) → element to make zero Aug(i,i) → pivot element factor → ratio used for elimination
Aug(j,:) = Aug(j,:) - factor*Aug(i,:);	Aug(j,:) → full row j Aug(i,:) → full pivot row Row operation: subtract scaled pivot row→ makes element below pivot = 0
end	Ends inner loop (rows below pivot processed)
end	Ends outer loop (all pivots processed)

Program Line	**Explanation**
x = zeros(n,1);	Creates solution vector [0;0;0]
for i = n:-1:1	Reverse loop → from last row to first
sum_val = Aug(i,i+1:n)*x(i+1:n);	Aug(i,i+1:n) → coefficients of known variables x(i+1:n) → already found values Matrix multiplication → gives sum of known terms
x(i) = (Aug(i,end) - sum_val)/Aug(i,i);	Aug(i,end) → RHS value sum_val → known terms Aug(i,i) → diagonal value Formula: (RHS − known)/diagonal
end	Ends back substitution loop

OUTPUT

Program Line	Explanation
disp('Solution is:')	Prints message
disp(x)	Displays solution vector

Experiment -9

Write a MATLAB program to find a root of the equation

$$f(x) = 2x^2 - 5x + 3 = 0$$

using the Newton–Raphson method.

- Take the **initial guess** as $x_0 = 0$
- Use a **tolerance** of 10^{-7}
- Perform iterations until convergence or up to 100 iterations
- **Display the root and number of iterations**

MATLAB Program	Explanation
f=@(x)x-(2*x^2-5*x+3)/(4*x-5);	Newton–Raphson iteration formula for given equation $x_{n+1} = x_n - \frac{f(x)}{f'(x)}$
x=0;	Initial guess
to1=1e-7;	Tolerance (accuracy)
for iteration=1:100	Loop (max 100 iterations)
xnew=f(x);	Compute next value
if abs(xnew-x)>to1	Check convergence
x=xnew;	Update value
else	If converged
break	Stop iteration
end	Ends the if-condition.
end	Ends them for-loop

MATLAB Program	Explanation
fprintf(‘Root=%f at iteration no.%d\n', xnew, iteration)	Displays the final root and number of iterations taken to converge.

fprintf → Prints formatted output to screen

'root=%f ...' → Text format string

%f → Displays the root value (in decimal form)

%d → Displays iteration number (integer)

xnew → Final computed root

iteration → Number of iterations taken

\n → Moves output to next line

Experiment -10

Write a MATLAB program to compute the **Discrete Fourier Transform (DFT)** of the sequence $x[n] = [1, 2, 3, 4]$ using the DFT formula

- Find the length of the sequence.
- Compute the DFT using nested loops.
- Display the output sequence.

What is Discrete Fourier Transform (DFT)?

- **DFT** converts a discrete-time signal (time domain) into its **frequency components (frequency domain)**.
- It tells **which frequencies are present** in the signal and their strengths.
- The mathematical formula of DFT is:

$$X[k] = \sum_{n=0}^{N-1} x[n] e^{-j\frac{2\pi}{N}kn}$$

Where:

- $x[n]$ = input sequence
- $X[k]$ = DFT output
- N = number of samples
- $k = 0, 1, 2, ..., N-1$

Program Statement	Explanation
x=[1,2,3,4];	Defines the input sequence.
N=length(x);	Finds the number of samples.
X=zeros(1,N);	Initializes output array with zeros.
for k=0:N-1	Outer loop for frequency index (k).
for n=0:N-1	Inner loop for summation index (n).
X(k+1)=X(k+1)+x(n+1)*exp(-1i*2*pi*k*n/N);	Computes DFT using the formula and stores the result.
end	Ends inner loop.
end	Ends outer loop.
display(X);	Displays DFT output.